HE-IFD: Privacy-Preserving One-Shot Federated Fine-Tuning of Pretrained Models under Homomorphic Encryption

Halil İbrahim Kanpak, Sinem Sav, Alptekin Küpçü

***Abstract*—Federated learning lets parties train a shared model without pooling their data, but its one-shot variant, which communicates in a single round, faces an unresolved privacy dilemma: sending a client's contribution in plaintext exposes it, while protecting it with differential privacy injects noise that degrades the released model and offers only a statistical, not a cryptographic, guarantee, and homomorphic encryption has so far been confined to multi-round encrypted training or to secure-aggregation primitives. We present HE-IFD, to our knowledge the first one-shot federated learning protocol whose contributions are protected by homomorphic encryption. Each client fine-tunes a small low-rank adapter on a frozen pretrained backbone for a bounded number of steps, and the server combines the resulting parameter displacements with a single sample-weighted linear operation under multiparty CKKS, at multiplicative depth one. Two design choices make this work: a frozen pretrained backbone supplies, for free, the shared frame a one-shot linear aggregation needs under heterogeneous data, so there is no alignment phase and nothing data-derived leaves a client before encryption; and freezing the adapter's down-projection makes the encrypted aggregate exact task arithmetic, halving the encrypted payload and keeping the circuit at depth one. Because a server computing under encryption is blind, we let it emit several depth-one candidate aggregates and have the clients select among them after decryption, which lifts the released model under heterogeneity and yields a measured defense against a poisoning client at no homomorphic cost. Across vision and language tasks, from frozen RoBERTa and ViT to a billion-parameter language model, the method produces a common model stronger than any client's own and, outside the most skewed regimes, close to a centralized one, while membership inference against the released model stays at or near chance. A real multiparty CKKS implementation reproduces the plaintext aggregate within the approximation bounds of CKKS at a few megabytes per round.**

***Index Terms*—Federated learning, one-shot federated learning, federated fine-tuning, homomorphic encryption, multiparty computation, privacy-preserving machine learning.**

I. Introduction

Many organizations would benefit from training a shared model on their combined data but cannot pool it, whether for legal, competitive, or privacy reasons. Federated learning addresses this by exchanging model updates instead of data, but in its usual form it does so over many rounds of communication, and each round discloses a fresh update computed on private data. One-shot federated learning removes the first cost by collapsing the entire interaction into a single upload and download. It does not, by itself, remove the second. A client's single contribution still encodes its private data, and the field's answers to this have been unsatisfying at the extremes: sending the contribution in plaintext exposes it, while protecting it with differential privacy injects noise into the released model and provides only a statistical, not a cryptographic, guarantee. Homomorphic encryption promises a way out by operating on contributions while they stay encrypted, but it has been applied to federated learning either as multi-round encrypted training, whose cost is measured in hours and gigabytes, or as a secure-aggregation primitive embedded in an iterative protocol. No existing method delivers a one-shot federated learning protocol whose contributions are protected cryptographically rather than by accuracy-degrading noise.

We present **HE-IFD**. Each client fine-tunes a small low-rank adapter and classifier head on a frozen public backbone, starting from a shared public initialization, and the server combines the clients' contributions with a single linear operation, $\theta^\star = \theta_0 + \sum_j w_j \Delta_j$, evaluated under multiparty CKKS. The server holds the shared initialization θ_0 as a public constant and never modifies it during the protocol: it only adds the aggregated displacement to this preserved model, so no training takes place on the server side. Because the combination is linear, it maps onto the operations a homomorphic scheme evaluates at multiplicative depth one, without bootstrapping, and moves tens of megabytes per round rather than hundreds of gigabytes. The privacy of the contributions is therefore cryptographic, and because the backbone is frozen and never enters the encrypted combination, the same protocol and the same cost carry across pretrained backbones in vision and language.

The difficulty a one-shot linear aggregation must overcome is that independently trained models do not average well under heterogeneous data: starting from different initializations and fitting different local distributions, their updates point in conflicting directions and cancel when summed. Fine-tuning a frozen pretrained backbone removes this difficulty without any alignment phase. The fixed backbone places every client in a common representation, and every client starts its small trainable unit from the same public initialization, so their bounded updates are expressed in one frame and reinforce rather than cancel. Where prior one-shot federated learning

Corresponding Author: Sinem Sav (sinem.sav@cs.bilkent.edu.tr)

Halil İbrahim Kanpak is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: hkanpak21@ku.edu.tr).

Sinem Sav is with the Department of Computer Engineering, Bilkent University, Ankara, Türkiye (e-mail: sinem.sav@cs.bilkent.edu.tr).

Alptekin Küpçü is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: akupcu@ku.edu.tr).

Manuscript received XX XX, XXXX; revised XX XX, XXXX.

must first agree on a starting point by exchanging per-class summaries (an extra round that exposes data-derived quantities and, when protected, spends a differential-privacy budget), a pretrained backbone supplies the shared frame without that step, and nothing data-derived leaves a client before encryption.

We show across vision and language tasks that the method produces a common model stronger than any client's own and, outside the most skewed regimes, close to a centralized model trained on the pooled data, while every client gains the ability to classify categories it never observed locally. We confirm in a real multiparty CKKS implementation that the encrypted aggregation reproduces the plaintext result within the approximation bounds of CKKS at tens of megabytes per round, and we measure with membership inference that the only quantity exposed in the clear, the released model, leaks little about any client's data.

Our contributions are as follows.

- We introduce, to our knowledge, the first one-shot federated learning protocol whose contributions are protected by homomorphic encryption: a single round whose only server operation is a linear aggregation of multiplicative depth one under multiparty CKKS, followed by threshold decryption (section III). All prior homomorphic federated learning is multi-round, and all prior one-shot federated learning is plaintext or differentially private.
- We identify a co-design that makes one-shot encryption cheap. Fine-tuning a frozen pretrained backbone supplies the shared frame a linear aggregation needs at no cost, and freezing the low-rank adapter's down-projection (training only the up-projection and head) makes the encrypted aggregate *exact* task arithmetic on the weight updates, with no bilinear cross-terms to resolve under encryption. The entire cryptographic object is then tens of ciphertexts, communicated once (sections III and III-D).
- We introduce encrypted multi-candidate release with client-side selection: the server emits several depth-one candidate aggregates (scaled, curvature- and coverage-weighted via separate numerator/denominator decryption, and leave-one-out), and the clients, after threshold decryption, select one by a weighted vote on their local data. This restores data-dependent aggregation choice to a cryptographically blind server at no homomorphic cost, lifts the released model under heterogeneity, and yields a measured robustness defense (sections III-F and IV-C).
- We give a privacy design in which the contributions are protected by cryptography alone, with no perturbation and no differential privacy, so the only quantity exposed in the clear is the final shared model, whose residual leakage we measure directly with membership inference (section IV-G); and we validate the encrypted aggregation in a real multiparty CKKS implementation across vision and language backbones at a cost of a few megabytes per round (sections IV-B and IV-F).

II. Related Work

The setting we study lies at the intersection of three lines of work that have largely developed in isolation: how federated learning communicates, how it is made private, and how it has been compressed into a single round. Tracing how each arrived at its present state makes clear both what is established and where the gap we address lies.

Federated learning began as an iterative procedure. In the original formulation a server repeatedly broadcasts a global model, clients improve it on their local data, and the server averages the returned updates [1]. This works well but inherits two persistent costs from its iterative nature. The first is communication: reaching a good model can take tens or hundreds of rounds, each moving a full model in both directions. The second is exposure: every round reveals a fresh update computed on private data, and a long sequence of such updates is a rich target for inference. Much of the subsequent literature can be read as attempts to control one of these two costs without worsening the other.

The exposure problem has been met with two broad tools, distinguished by whether they alter the model that is ultimately released. Secure aggregation lets a server learn only the sum of the clients' updates and nothing about any individual one, using masking protocols that cancel in the aggregate [2]; it is lossless, in that the aggregate is exact, but it protects only the per-round sum and still operates within the iterative protocol. Differential privacy instead perturbs the updates themselves, most commonly through differentially private stochastic gradient descent [3]; the guarantee is strong and composable, but because the noise enters the released model it trades privacy directly against accuracy. This distinction, between privacy that costs accuracy and privacy that does not, is the axis along which our own privacy design is best understood, and it recurs throughout what follows.

Homomorphic encryption offers a third route, protecting the updates by computing on them while they remain encrypted. In its most ambitious form it has been used to train a neural network entirely under encryption across many parties. POSEIDON, for instance, runs encrypted stochastic gradient descent under multiparty CKKS, with the nonlinear activations replaced by polynomial approximations so that they can be evaluated homomorphically [4]. The privacy adds no accuracy cost and the result is strong, but the approach remains fundamentally iterative and pays for the encrypted nonlinearities with a deep multiplicative circuit, repeated bootstrapping, and a wall-clock measured in hours. A lighter line keeps the encryption but narrows its job to aggregation alone: schemes that combine multi-key CKKS with masking reduce a round of secure summation to a single non-interactive client-to-server transmission [5]. These are efficient and genuinely cryptographic, but they encrypt the sum of model updates within an iterative training protocol rather than a one-shot contribution, and they perform aggregation rather than knowledge transfer. On the encryption axis these works are the closest to ours, and the contrast is precise: they are either multi-round or are secure-aggregation primitives, whereas we encrypt a single fine-tuning displacement.

Homomorphic encryption has also been applied to federated learning purely for aggregation, encrypting the per-round updates so the server can sum them without seeing any in the clear; BatchCrypt, FedML-HE, and FedSHE develop this

for cross-silo training with various packing and quantization optimisations [6], [7], [8], and a recent line encrypts only the low-rank adapters of a parameter-efficient fine-tune: SHE-LoRA selectively encrypts a sensitivity-chosen subset of the adapter under CKKS each round [9]. These remain multi-round and, like secure aggregation, protect the sum rather than transferring knowledge; encrypting the adapter rather than the full model narrows the per-round object, but the cost is still paid on every round of an iterative protocol, and the bilinear coupling of the two adapter factors (discussed below) is left to be resolved under encryption. Our protocol differs on both axes: a single round, and an adapter parameterisation for which the aggregate is exactly linear, so no coupling remains to resolve under encryption. A recurring obstacle across all encrypted-training work is the nonlinearity: activations must be replaced by polynomial approximations to be evaluated homomorphically, which deepens the multiplicative circuit and is delicate to train, as a line of work on polynomial activations and encryption-friendly networks documents [10], [11], [12], [13], [14]. Our design sidesteps this entirely: the only encrypted operation is linear, so no activation is ever evaluated under encryption and the circuit stays at depth one.

A separate line federates *parameter-efficient fine-tuning*, communicating only a low-rank adapter rather than the full model. FedIT first applied this to instruction tuning by averaging the adapters round by round [15], but naive per-factor averaging is biased: a client learns an update $\Delta W = BA$, and averaging the factors separately yields $(\sum_j B_j)(\sum_j A_j) \neq \sum_j B_j A_j$, a discrepancy formalised as aggregation noise by FLoRA, which removes it by stacking the clients' factors [16]; FlexLoRA instead reconstructs and averages the full products before redistributing by SVD [17], HetLoRA reconciles heterogeneous ranks [18], and FedSA-LoRA shares only one factor [19]. The fix we adopt is FFA-LoRA, which freezes the randomly initialised down-projection and trains only the up-projection, so that averaging the trained factor is exact; it was introduced to stabilise multi-round, differentially private federated tuning [20]. Under encryption this choice has a sharper consequence than in plaintext: the bilinearity is not merely a bias to correct but a cost cliff, since resolving it on the server would require ciphertext-by-ciphertext products, whereas freezing the down-projection makes the one-shot encrypted aggregate exact at multiplicative depth one. Our aggregation is, algebraically, task-vector merging [21], which has been shown equivalent to one-shot federated averaging [22]; that a single round of fine-tuning suffices for foundation models is itself established in plaintext [23]. What none of these provide is cryptographic protection of the contributions, which is our subject.

The communication problem has driven the parallel development of one-shot federated learning, which collapses the entire exchange into a single upload and download [24], [25]. The earliest approaches transferred knowledge rather than weights, having clients exchange predictions on a shared public set so that heterogeneous models could teach one another [26], [27]; in practice these remained iterative, repeating the exchange to converge. Ensemble distillation made the distillation step explicit by training a server model against the clients' aggregated predictions, though in its standard configuration it too runs for many rounds [28]. The decisive step to a true single round came with data-free one-shot methods, which avoid any shared data by having the server distil the clients' models through a generator or through synthesised inputs, as in DENSE and its successors [29], [30], and with fusion-based methods that stitch client models together with lightweight learned adapters [31] or aggregate them through a layer-wise posterior [32], [33]. These methods are the closest to ours on the one-shot and distillation axes, and they establish that a single round can produce a usable joint model. What they do not provide is any cryptographic protection: they operate in plaintext, so the contributions each client sends are exposed.

Closing that last gap, a handful of works add privacy to one-shot federated learning, and they do so with differential privacy. FedAUXfdp distils through a pretrained feature extractor on auxiliary public data and releases the client contributions under a differentially private mechanism [34]; related approaches protect a diffusion-generated [35] or a teacher-ensemble [36] surrogate, or add noise-free differential privacy to the distillation itself [37], under the same kind of guarantee. These are the natural quantitative peers for our method, since they share its one-shot, distillation-based structure, but their privacy is lossy in exactly the sense above: the noise needed for a meaningful budget is added to the artifact that is released, and accuracy falls as the budget tightens.

The reason privacy is the binding concern here is that the artifacts federated protocols exchange leak. Sharing gradients or model updates permits reconstruction of training inputs [38], [39], [40] and supports membership and property inference [41], [42], [43], [44], [45], [46], and even released confidences enable model inversion [47], [48]. Federated distillation, which shares predictions or a public-data signal instead of gradients, was hoped to sidestep this, but a growing body of attacks shows it leaks as well: paired-logit inversion recovers images from shared logits [49], and membership can be inferred from distillation outputs and from public-dataset usage [50], [51], [52], [53], [54]. This is why we protect the contributions cryptographically rather than by perturbation, and why we measure in section IV-G what the released model still reveals.

Seen together, the two strongest lines never meet. The cryptographic work that would protect contributions without loss is either multi-round encrypted training or a secure-aggregation primitive, and the one-shot work that communicates in a single round is either plaintext or made private only by lossy perturbation. The method we present occupies the empty intersection: a one-shot federated learning protocol whose single encrypted operation is a linear aggregation under multi-party homomorphic encryption, giving the accuracy-preserving privacy of the cryptographic line at the communication budget of the one-shot line.

III. Method

A. Overview

We consider N clients that hold private labelled data over a common label space of C classes and want a shared classifier

without pooling their data. Client j holds $\mathcal{D}_j$ of size n_j; the data are heterogeneous, with each client's class proportions drawn from a Dirichlet distribution of concentration α (smaller α means more skew). Two constraints separate our setting from ordinary federated learning. The protocol is *one-shot*: one upload and one download per client, with no iterative exchange. And privacy is *cryptographic*: a client's contribution is never revealed in the clear, and we are unwilling to accept the accuracy loss that differential privacy incurs when it is the only line of defence.

These two constraints largely determine the design. Under encryption the server is blind: it operates on ciphertexts that are by construction indistinguishable from random, so it cannot read a value, compare two values, or branch on the data, and every multiplication it performs spends part of a finite noise budget. The only computation it can afford at scale is a shallow, fixed, linear one (a weighted sum), and a weighted sum of independently trained models is meaningful only when those models already share a frame. A from-scratch federation has no such frame, and building one would take either several rounds or server-side computation on the models themselves, neither of which encryption allows. A *pretrained* model supplies the frame directly: freezing a public backbone fixes the representation every client sees, so the small part each client trains starts from the same place and stays close. Fine-tuning a pretrained model is therefore not a convenience but the learning setting this protocol admits, and it removes the need for any alignment step before training, the property that most distinguishes HE-IFD from prior one-shot federated learning, which spends an extra communication phase, and often a differential-privacy budget, to bring clients into a common frame before they begin.

Concretely, every client uses a frozen public backbone φ followed by a small trainable unit: low-rank adapters together with a classifier head, with parameters ψ. A low-rank adapter writes the update to a weight matrix as a product $\Delta W = BA$ of two small factors; we freeze the down-projection A at its shared public initialisation and train only the up-projection B and the head. This single choice is what makes the encrypted aggregation *exact*: with A fixed and common to all clients, a weighted sum of the trained factors reconstructs the weighted sum of the updates, $\big(\sum_j w_j B_j\big)A = \sum_j w_j B_j A$, whereas training both factors would make the update bilinear and the per-factor average a different, and worse, quantity. The backbone never changes, is never transmitted, and is never encrypted; only the trainable unit is fine-tuned, communicated, and operated on under encryption. Keeping that unit small is what makes the encrypted step inexpensive: it is a tiny fraction of the backbone (a low-rank adapter and head, on the order of 10^5 parameters against a backbone of 10^8), so a client's entire encrypted contribution is a few tens of ciphertexts rather than the model itself. Every client initializes the trainable unit to the *same* public constant θ_0, the standard adapter initialization, agreed in advance and carrying no client data. There is no prototype exchange and no peer-to-peer phase; nothing data-derived ever leaves a client before encryption.

The protocol is two steps, shown in fig. 1. Starting from the shared initialization θ_0, each client fine-tunes its trainable unit on its own data for a bounded number of steps and records the displacement it travelled, $\Delta_j = \theta_j^{(K)} - \theta_0$. The server then combines the encrypted displacements into a single global model, $\theta^\star = \theta_0 + \sum_j w_j \Delta_j$, using only homomorphic additions and multiplications by public scalars, after which a threshold of clients jointly decrypts the result.

What we are after is not a model that beats each client on that client's own dominant classes; a local specialist on a narrow slice of the label space is difficult to surpass on that slice, and we do not claim to. We are after a common model every party can use, that classifies well across the whole label space including classes a given client never observed locally, and that is produced without any party exposing its data or its update in the clear.

Algorithm 1 One-shot federated fine-tuning with encrypted aggregation. $\mathsf{Enc}(\cdot)$ denotes encryption under the clients' collective CKKS key.

Require: clients $1, \dots, N$ with data $\mathcal{D}_j$; frozen backbone φ; public init θ_0; steps K

clients run distributed key generation, yielding a collective CKKS key

Local fine-tuning (each client j in parallel)

$\theta_j^{(0)} \leftarrow \theta_0$

for $k = 1$ to K **do**

  $\theta_j^{(k)} \leftarrow \theta_j^{(k-1)} - \eta \nabla \mathcal{L}\big(f_\theta; \mathcal{D}_j\big)$ ▷ supervised fine-tuning of the trainable unit

end for

$\Delta_j \leftarrow \theta_j^{(K)} - \theta_0$; upload $\mathsf{Enc}(\Delta_j)$

Encrypted aggregation (server)

$\mathsf{Enc}(\theta^\star) \leftarrow \theta_0 + \sum_j w_j \cdot \mathsf{Enc}(\Delta_j)$ ▷ PT×CT, CT+CT; depth 1

broadcast $\mathsf{Enc}(\theta^\star)$

Threshold decryption (clients)

a threshold of clients jointly decrypt $\mathsf{Enc}(\theta^\star)$ to recover $\theta^\star$

TABLE I
NOTATION.

Symbol	Meaning
N	number of participating clients
$\mathcal{D}_j,\ n_j$	client j's local dataset and its size $n_j = \lvert\mathcal{D}_j\rvert$
C	number of classes
α	Dirichlet concentration (label heterogeneity)
φ	frozen public backbone (never trained or encrypted)
ψ	parameters of the trainable unit (adapter + head)
f_θ	model on the frozen features, parameters θ
θ_0	shared public initialization of the trainable unit
K	local fine-tuning steps
Δ_j	displacement $\theta_j^{(K)} - \theta_0$
w_j	sample weight $n_j / \sum_i n_i$
$\theta^\star$	aggregated global model $\theta_0 + \sum_j w_j \Delta_j$

B. Notation

Table I collects the symbols used throughout. Client indices are $j \in \{1, \dots, N\}$.

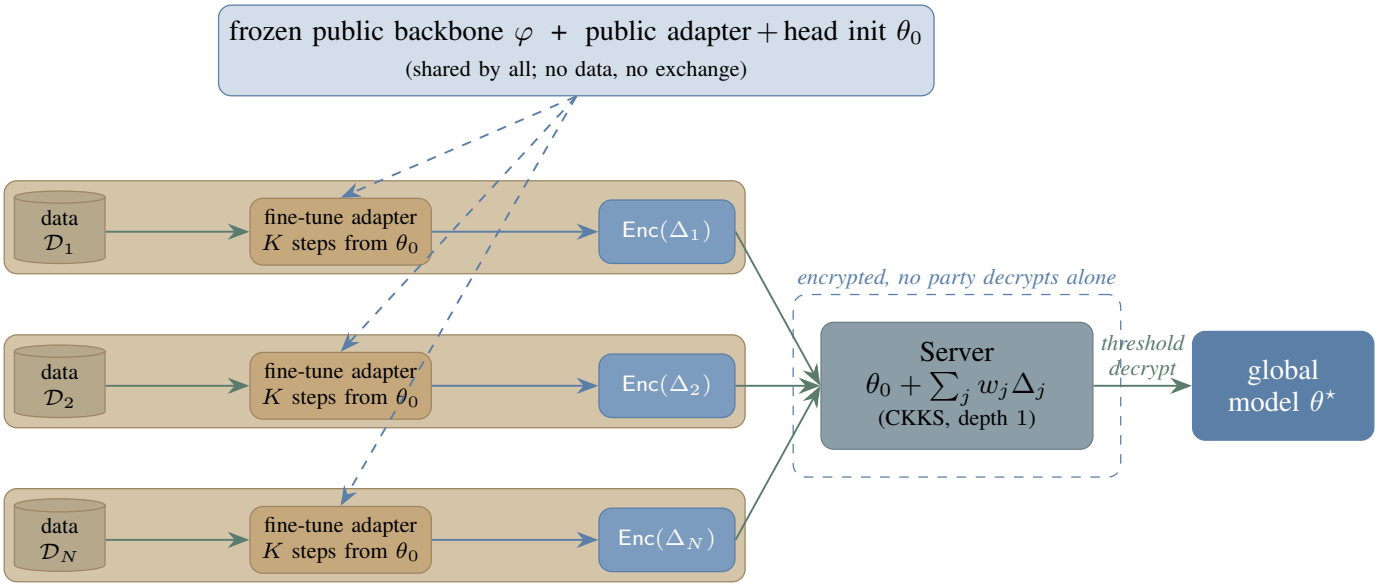

Fig. 1. HE-IFD. A frozen public backbone φ and a public initialization θ_0 of the small trainable unit (low-rank adapter and head) are shared by all clients and carry no data, so there is no alignment phase; the backbone is never trained or encrypted. Each client fine-tunes the adapter on its own private data $\mathcal{D}_j$ for a bounded number of steps from θ_0 and encrypts the resulting displacement $\Delta_j = \theta_j - \theta_0$. The server's sole operation is the sample-weighted linear combination $\theta^\star = \theta_0 + \sum_j w_j \Delta_j$ under multiparty CKKS (additions and public-scalar multiplications only; multiplicative depth one); it never decrypts. A threshold of clients jointly decrypts the global model $\theta^\star$.

C. Multiparty Homomorphic Encryption

We build on the CKKS scheme [55], a ring-learning-with-errors construction [56] that performs approximate arithmetic on vectors of real numbers under encryption. CKKS packs many real slots into one ciphertext and supports two homomorphic operations of interest here: addition of two ciphertexts, and multiplication of a ciphertext by a plaintext. It is a levelled scheme, meaning each multiplication consumes one level of a finite budget; our protocol uses only additions and multiplications by public plaintext scalars, so it consumes a single level and never requires the expensive bootstrapping step [57] that refreshes a depleted budget.

A single-key scheme would require one party to hold the secret key and therefore be trusted with decryption. We instead use the multiparty variant of CKKS [58], in which the secret key is shared among the clients through a distributed key-generation protocol that produces a collective public key without ever materialising the secret in one place. Any party, including the server, may compute on ciphertexts under the collective public key, but decryption requires the cooperation of a threshold of clients, carried out as a collective key-switching protocol. No single party, and in particular not the server, can decrypt on its own. This is exactly the property our aggregation needs: the server performs the homomorphic computation on encrypted contributions, yet only the clients, acting together, can reveal the result.

D. The Shared Frame

The natural one-shot recipe (let each client train independently and have the server average the results) fails under heterogeneity. When clients begin from different initializations and fit different skewed distributions, their trained parameters occupy different regions of parameter space and their average lies in none of them: the updates point in conflicting directions and cancel when summed. The remedy is for every client to begin from the same point and move only a little, so their displacements live in one frame and add constructively rather than fighting one another.

A pretrained backbone provides this directly. Because the representation is fixed and shared, the trainable unit lives in the same parameter space for every client, and initializing it to a common public constant θ_0 places all clients at the same point in that space before any training. No data is consulted to build θ_0 and nothing is exchanged to agree on it; it is a fixed, public initializer. This is exactly the step a from-scratch federation cannot take and must replace with a data-dependent alignment phase, a phase that costs an extra round, exposes a per-client summary, and, when that summary is protected, spends a differential-privacy budget. Fine-tuning a pretrained model removes the phase, and with it the only channel through which client data would otherwise leave in the clear before encryption. We verify in section IV that this shared start is what makes the one-shot combination work (averaging independently trained units without it collapses), and that the frozen backbone supplies the start with no alignment phase.

The frame the backbone supplies is the representation, not the classifier. The shared initializer θ_0 is an untrained head: it places every client at the same point in parameter space but encodes nothing about any class. A client therefore contributes a useful displacement for a class only when it holds examples of that class, so the global model's accuracy on a class is bounded by how many clients cover it. This is the one accuracy cost of dropping alignment, and it is a property of *coverage* rather than of the frame. It is most visible when the label space is large and the skew extreme, so that each client sees only a

sliver of the classes, and it narrows as each client sees more of the label space, closing well before the data become uniform. We characterize this gap directly in section IV, measured against an alignment-based reference that closes it only by exposing per-client class summaries in the clear, the exposure the cryptographic design exists to remove.

E. Local Fine-Tuning

Starting from θ_0, client j fine-tunes its trainable unit on its own data for K steps of ordinary supervised training, producing parameters $\theta_j^{(K)}$. The step budget K is deliberately small. This is not an optimisation shortcut but part of the method: a short trajectory keeps the trainable unit close to θ_0, and therefore inside the shared frame, so the displacement it produces is still expressed in the common coordinates. Allowed to train to convergence, each client would drift to its own local optimum and reintroduce exactly the conflict the shared start was meant to avoid.

At the end of its trajectory the client transmits not its parameters but the displacement

$$\Delta_j \;=\; \theta_j^{(K)} - \theta_0, \tag{1}$$

the distance it travelled from the shared start. Because every client departs from the same θ_0, these displacements are directly comparable across clients, which is what makes the linear combination that follows meaningful. A variant that distils a locally trained teacher into the unit, rather than fine-tuning on hard labels directly, fits the same template and is evaluated as an ablation in section IV.

F. Encrypted Linear Aggregation

The clients hold a joint encryption key produced by distributed key generation, so no single party, and in particular not the server, ever holds the secret key. Each client encrypts its displacement Δ_j; since the trainable unit is small relative to the backbone, this is a few tens of ciphertexts. The server fuses the per-client displacements into a single update of the shared model by the sample-weighted combination

$$\theta^\star \;=\; \theta_0 + \sum_{j=1}^{N} w_j\,\Delta_j, \qquad w_j = \frac{n_j}{\sum_i n_i}, \tag{2}$$

in which each client contributes in proportion to the data n_j behind it. Only the displacements are encrypted; the weights w_j and the initializer θ_0 are public and stay in plaintext. Forming eq. (2) therefore requires only multiplying each ciphertext by the public scalar w_j and adding the results, with θ_0 added in plaintext: there is no ciphertext-by-ciphertext multiplication, no encrypted optimizer state, and no bootstrapping, so the circuit has multiplicative depth one. A threshold of clients then jointly decrypts $\theta^\star$ by collective key-switching, and the server never observes a plaintext update. This is the central design choice: casting aggregation as an operation that is *linear in the encrypted quantities* yields a server computation that maps onto the least costly operations a levelled scheme provides, without the encrypted nonlinearities that make encrypted training expensive. The operations are also fully parallel (the clients fine-tune and encrypt independently, and the server's sum is an associative reduction), so the end-to-end wall-clock is one client's local computation plus a single aggregation, independent of N. We instantiate eq. (2) in a real multiparty CKKS implementation and report its measured correctness and cost in section IV-F.

It is worth being precise about why this works, because eq. (2) is easily misread. Since the weights sum to one, the displacement form is algebraically a weighted average of the clients' fine-tuned units, $\theta^\star = \sum_j w_j\,\theta_j^{(K)}$, which is exactly task-vector merging [21]. That linearity is what keeps the operation inexpensive to encrypt, and it is exact rather than approximate only because the adapter's down-projection is frozen: were both adapter factors trained, the per-client update B_jA_j would be bilinear, the average of the factors would not equal the average of the updates, and the quantity the server can cheaply compute would no longer be the one the method needs. But the same average over units trained independently, from different starts, to convergence is the naive combination that fails above. What makes the average land in a good region here is the protocol around it: every $\theta_j^{(K)}$ starts from the shared θ_0 and moves only a bounded distance, so all of them stay in the common frame and their weighted average is itself a sensible model. The contribution is not a new aggregation rule but a one-shot protocol under which a linear, and therefore inexpensively encryptable, aggregation produces a strong shared model.

G. Multi-Candidate Release and Client-Side Selection

A server computing under encryption is blind: it cannot inspect a value, so it cannot adapt the aggregation rule to the data the way a plaintext aggregator could. We recover that adaptivity without giving the server any power to read, by moving the choice to the clients after decryption. The key observation is that several useful aggregation rules are each, on their own, a depth-one linear function of the encrypted displacements, so the server can evaluate *all of them* in one pass and let the clients pick. We form a small set of candidate aggregates:

- a *scaling family* $\theta_0 + \lambda \sum_j w_j\Delta_j$ for a public grid of λ, which trades the shared start against the aggregated move and costs nothing extra, since all candidates are affine in the same encrypted sum;
- *reweighted merges* $\theta_0 + (\sum_j w_j g_j \odot \Delta_j) \oslash (\sum_j w_j g_j)$, where g_j is a public-after-decryption per-coordinate weight (a diagonal Fisher estimate, or a per-class example count applied to the head rows). Each client forms the products $g_j \odot \Delta_j$ and g_j *before* encryption, the server adds the two ciphertext stacks, and the clients divide the decrypted numerator by the decrypted denominator in plaintext, so the nonlinear division never occurs under encryption;
- *leave-one-out* aggregates $\theta_0 + \sum_{i\neq k} \tilde{w}_i\Delta_i$, one per excluded client k, with public renormalised weights $\tilde{w}_i$, used for robustness (section IV-C).

The server emits the encrypted candidates; a threshold of clients jointly decrypts each one; and every client scores the

decrypted candidates on a small held-out slice of its own data, the federation releasing the sample-weighted plurality choice. Selection is therefore post-decryption and entirely client-side: it adds one threshold decryption per candidate and no homomorphic multiplications beyond the depth-one aggregate. The candidates differ in what they reveal, and we account for this honestly. The scaling family is collinear, $\theta^\star(\lambda) = (1-\lambda)\theta_0 + \lambda\theta^\star(1)$, so disclosing the whole grid reveals nothing beyond the released model itself; the reweighted numerator and denominator reveal two sums rather than their ratio; and a leave-one-out pair isolates a single client's contribution. All three are admissible under the threat model of section III-H, in which participants legitimately hold the released model and the server still sees only ciphertexts; a deployment unwilling to expose per-client contributions to fellow participants restricts the candidate set to the scaling family, which leaks nothing further.

H. Threat Model

The participants are N clients and a single aggregation server. The server is honest-but-curious: it follows the protocol but may try to infer private information from anything it observes. During aggregation it sees only ciphertexts under the collective key, and as established in section III-C it cannot decrypt them. A minority of clients may likewise be honest-but-curious and may collude, but any coalition below the decryption threshold learns nothing about another client's displacement, since recovering a plaintext requires the cooperation of the threshold. This is a stronger position than secure-aggregation protocols, where the revealed sum has itself been shown to leak across rounds [59], [60], because in our one-shot protocol the server never obtains a plaintext sum at all.

We are explicit about one boundary of this model. Every participant obtains the released model $\theta^\star$ in the clear after threshold decryption, by design, since the purpose of the protocol is to hand the parties a shared model. Any inference a participant then performs on $\theta^\star$ against another participant's data is therefore not an attack the protocol can prevent but a property of releasing a useful shared model at all, and we measure it directly in section IV-G under exactly this fellow-participant adversary. The cryptographic guarantee concerns the *contributions*: the server and any sub-threshold coalition observe only ciphertexts. This is the sense in which the protection target differs from differentially private one-shot methods, which perturb the released artifact to protect it against all parties, including participants, at a cost to its accuracy. Robustness to actively malicious or Byzantine clients is outside the cryptographic guarantee; the multi-candidate mechanism of section III-G provides a lightweight measured defense (section IV-C), and stronger guarantees we discuss as future work in section IV-K.

I. Security

The model updates are protected cryptographically: the aggregation of eq. (2) runs entirely under multiparty CKKS with threshold decryption, so the displacements Δ_j are never exposed to the server or to any minority of clients, and they are protected without any perturbation. The cryptographic layer adds no accuracy cost. This is the central privacy claim, and the point on which the method departs from differentially private one-shot federated learning, which injects noise into the updates it ultimately releases and so pays a utility tax on the very thing it ships.

We make this precise. Let the view of the honest-but-curious server be everything it receives: the public collective key, the public weights $\{w_j\}$ and initializer θ_0, the client ciphertexts $\{\mathsf{Enc}(\Delta_j)\}_{j=1}^N$, and the homomorphically computed $\mathsf{Enc}(\theta^\star)$. We write $\mathsf{view}_{\mathrm{srv}}(\{\mathcal{D}_j\})$ for this view as a function of the private datasets.

Proposition 1. *Under the IND-CPA security of the multiparty CKKS scheme and its t-out-of-N threshold decryption, for any honest-but-curious server colluding with up to $t-1$ clients there is a probabilistic polynomial-time simulator $\mathcal{S}$, given only the public parameters, such that $\mathsf{view}_{\mathrm{srv}}(\{\mathcal{D}_j\}) \overset{c}{\approx} \mathcal{S}$. In particular the server's view is computationally independent of the private datasets $\{\mathcal{D}_j\}$ and the displacements $\{\Delta_j\}$, and the server cannot recover any plaintext, including $\theta^\star$, without a decryption quorum.*

Proof sketch. The simulator outputs the public parameters together with N encryptions of zero under the collective key. By IND-CPA each genuine $\mathsf{Enc}(\Delta_j)$ is computationally indistinguishable from $\mathsf{Enc}(0)$; a hybrid over the N ciphertexts replaces them one at a time, so the whole collection is indistinguishable from the simulated one and carries no information about $\{\Delta_j\}$ or the data they derive from. The aggregate $\mathsf{Enc}(\theta^\star)$ is a deterministic function of those ciphertexts and the public $\{w_j\}, \theta_0$, so it adds nothing. Decryption needs t shares of a secret key that is never reconstructed in one place; the server with at most $t-1$ clients holds too few shares to decrypt. □

Proposition 1 pins down where information can flow. Because there is no alignment phase, no data-derived quantity leaves a client before encryption, so the protocol exposes private information through exactly one channel, intrinsic to the task rather than an artifact of our construction: the released model itself. After threshold decryption every client holds $\theta^\star$ in the clear, and white-box access to the jointly built model is unavoidable in any protocol whose purpose is to hand the parties a shared model. The residual information that model carries about training data is what we quantify with membership inference in section IV-G.

IV. Experiments

We evaluate the protocol's central claim: that encryption protects each client's contribution at no cost to the shared model's accuracy, where one-shot differentially private federated learning must trade accuracy for the same protection. The evaluation tests that claim and the design choices behind it. We ask, in order: what do fine-tuning the pretrained layers and selecting among depth-one candidate aggregates add over the naive one-shot average; how the method behaves under a poisoned client and under the trajectory budget; what it

TABLE II
EVALUATION AXES. EACH VARIES ONE DIMENSION AROUND THE DEFAULT ($N = 10$, RANK-8 ADAPTER, $K = 200$, THREE SEEDS); NO PUBLIC DATA IS USED ANYWHERE.

Axis	Setting	Question it probes
Trainable unit	head-only vs. rank-$\{4, 8, 16\}$ adapter	value of fine-tuning pretrained layers
Heterogeneity α	0.05 (severe) to 1.0 (mild)	robustness to label skew
Clients N	10, 20, 50, 100	scalability in participants
Trajectory K	100, 200, 400	the bounded-step budget

reaches at the published setups of prior one-shot methods and at billion-parameter scale; and what the cryptographic protocol costs in time, communication, and residual leakage. Throughout, the question is not whether the global model attains the highest possible test accuracy, but whether the clients jointly obtain a single model close to a centrally fine-tuned one, in one round, with cryptographic privacy that adds no accuracy cost.

A. Setup

a) No public data.: Every client starts from the *same* public pretrained backbone with a zero-initialized low-rank adapter and a freshly initialized head, the standard adapter initialization θ_0, which carries no client data and classifies at chance. Each client fine-tunes the adapter and head on its own private data for a bounded K steps and uploads the encrypted displacement; the server forms the depth-one weighted sum of eq. (2). No labelled public set is used anywhere; section IV-E discusses the easier case in which a deployment does have one.

b) Tasks and backbones.: We evaluate on four text classification tasks of increasing label-space size: AG-News (4 classes), TREC (6), DBpedia (14), and Banking77 (77), with two frozen encoders, RoBERTa-base and MPNet, and on CIFAR-100 with a ViT vision backbone. Data are partitioned across clients by a Dirichlet distribution over labels with concentration α (the standard label-skew partition: a smaller α leaves each client a few dominant classes and almost none of the rest). Unless stated otherwise there are $N = 10$ clients, the adapter is rank 8, the trajectory is $K = 200$ steps, and every number is averaged over three seeds.

c) What we report.: We report the test accuracy of the HE-IFD global model; the shared start θ_0 classifies at chance, since it carries no task head. Two reference points frame it: a frozen-feature head (a linear probe trained under the same protocol) and, as the ceiling, the same model fine-tuned centrally on the pooled private data, what one party would obtain holding all of it. The two quantities of interest are the *increment* of HE-IFD over the frozen-feature head and the *gap* below the centralized ceiling; the centralized accuracy itself we give in the relevant captions rather than repeat in every row. Table II lists the axes.

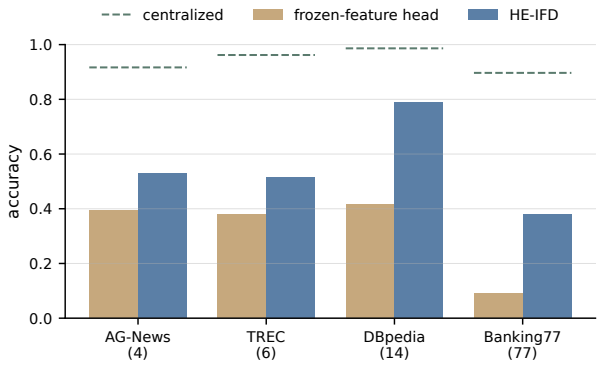

Fig. 2. HE-IFD lifts the single encrypted aggregation well above the chance start and above the naive sample-weighted average; the gap to centralized grows with the size of the label space, and no client data is shared.

TABLE III
HE-IFD AGAINST THE NAIVE ONE-SHOT AGGREGATION AND THE CENTRALIZED CEILING ($N = 10$, $\alpha = 0.1$, RANK-8 FROZEN-PROJECTION ADAPTER, THREE SEEDS). "NAIVE" IS THE PLAIN SAMPLE-WEIGHTED AVERAGE (λ=1, NO CANDIDATE SELECTION); HE-IFD IS THE CLIENT-SELECTED CANDIDATE; $\pm$ IS THE SEED STANDARD DEVIATION; THE GAP IS HE-IFD TO CENTRALIZED. TEXT TASKS USE FROZEN ROBERTA, CIFAR-100 A FROZEN VIT-B/16.

Task (C)	naive avg	HE-IFD	centralized	gap
AG-News (4)	0.51	**0.75**$\pm$0.09	0.91	0.16
TREC (6)	0.51	**0.72**$\pm$0.05	0.95	0.23
DBpedia (14)	0.80	**0.93**$\pm$0.01	0.99	0.06
Banking77 (77)	0.39	**0.77**$\pm$0.02	0.88	0.11
CIFAR-100 (100)	0.47	**0.78**$\pm$0.01	0.87	0.09

B. The Value of Fine-Tuning the Pretrained Layers

The design rests on two choices working together: adapting the backbone's own layers with a small frozen-projection adapter (small enough to encrypt inexpensively, yet fine-tuning the pretrained representation itself), and selecting among depth-one candidate aggregates after decryption rather than committing to a fixed rule the blind server cannot adapt. Figure 2 and table III measure both at $N = 10$ and moderate heterogeneity, against the naive sample-weighted average and the centralized ceiling.

Two effects compound in table III. First, fine-tuning the frozen backbone's own layers with a small adapter lifts the model far above the chance start: from a θ_0 that classifies at chance to between 0.72 and 0.93 across the four text tasks and 0.78 on CIFAR-100, with only the small adapter and head entering the encrypted combination, so the protocol and its cost do not change across tasks. Second, the candidate set with client-side selection (section III-G) lifts the released model well above the naive sample-weighted average, by $+0.24$ on AG-News, $+0.21$ on TREC, $+0.13$ on DBpedia and $+0.38$ on Banking77; the naive average is exactly the combination that the heterogeneity makes fragile, and the vote recovers the candidate that survives it. The remaining gap to the centralized fine-tune is small at moderate label spaces (0.06 on the 14-class DBpedia) and is largest where a single round must cover many classes each client barely observes; even there, freezing the adapter's projection and selecting a coverage-weighted

TABLE IV
ROBUSTNESS TO ONE POISONED CLIENT (DBPEDIA AND AG-NEWS, $N = 10$, $\alpha = 0.1$, THREE SEEDS). "PLAIN" IS THE UNDEFENDED SAMPLE-WEIGHTED AGGREGATE THAT INCLUDES THE ATTACKER; "HE-IFD" IS THE VOTE-SELECTED LEAVE-ONE-OUT CANDIDATE; "ORACLE" IS THE ATTACKER-FREE AGGREGATE; "EXCLUDED" COUNTS HOW OFTEN THE VOTE DROPPED THE ATTACKER.

Attack	plain	HE-IFD	oracle	excluded
sign-flip	0.25	**0.65**	0.65	6/6
large-Gauss	0.50	**0.67**	0.65	5/6
label-flip	0.48	**0.65**	0.65	6/6

candidate hold the 77-class Banking77 gap to 0.11, down from over 0.5 for the naive average. HE-IFD produces the model in one round, with no client exposing its data and with encryption that perturbs nothing.

C. Robustness to a Poisoned Client

The same multi-candidate mechanism that selects the best aggregation rule also gives a measured defense against a misbehaving client, at no homomorphic cost. We add to the candidate set the N leave-one-out aggregates (section III-G) and let one client (the one holding the largest shard, the worst case for sample weighting) upload a crafted displacement instead of an honest one: a sign-flipped update, a large-norm Gaussian, or one trained on label-flipped data. The honest clients still vote on their local holdouts, now over the plain aggregate and the N leave-one-out aggregates; the candidate that excludes the attacker wins whenever excluding it helps.

Table IV reports the result on DBpedia and AG-News at $N = 10$, three seeds. The undefended plain aggregate is pulled down to 0.25–0.50 depending on the attack, while the vote-selected model recovers to within rounding of the attacker-free oracle, and the vote identifies and excludes the attacker in 17 of 18 cells. The one miss is a Gaussian attack whose corruption was mild enough that including the attacker still scored best on the holdouts, i.e. the vote kept it precisely because it was not harmful. This is not a general Byzantine guarantee (it assumes an honest holdout majority and a single attacker), but it shows the candidate machinery already absorbs a poisoning client that the blind server cannot filter.

D. Trajectory Length

The trajectory length sets how far each client moves from the shared start before its displacement is aggregated. Table V varies the step budget K on DBpedia: within the swept range a longer trajectory helps monotonically, lifting HE-IFD from 0.91 at $K = 100$ to 0.94 at $K = 400$ and narrowing the gap to centralized to 0.05, the frozen backbone keeping even the longest trajectory's displacements coherent enough to sum.

E. When Public Data Is Available

We study the harder setting in which no public data exists, because that is where a one-shot encrypted aggregation is actually needed. When a deployment does have a public labelled set, the problem is strictly easier and standard recipes apply directly: the shared initialization can be warm-started on it, or a model can be trained on the public data and then refined. With enough of it, training a model from scratch on the public set is itself an option. We therefore treat public data as outside the method and rely on none of it anywhere in this paper.

TABLE V
TRAJECTORY LENGTH: HE-IFD ACCURACY (DBPEDIA, $N = 10$, $\alpha = 0.1$); CENTRALIZED ≈ 0.99.

K	HE-IFD	gap
100	0.91	0.08
200	0.93	0.06
400	0.94	0.05

F. Homomorphic-Encryption Implementation and Cost

We do not simulate the cryptography. We implement the entire protocol end to end in real multiparty CKKS on the Lattigo library [61], [58]: distributed key generation, encryption under the joint public key, the homomorphic weighted aggregation of eq. (2), and threshold decryption by a collective key-switch, and the decrypted aggregate matches the plaintext computation to within CKKS encoding precision (verified below). The server's only cryptographic operation is that depth-one weighted sum, and its cost is set entirely by the number of *fine-tuned* parameters that enter a ciphertext, not by the frozen backbone behind them. We measure the per-operation rates directly and calculate every reported configuration from them, so a small adapter and head are inexpensive to encrypt whatever model they sit on.

a) Parameters and setup.: All figures use ring degree 2^{14} (8192 slots per ciphertext), a modulus chain $\log Q = \{55, 45\}$ with key-switch prime $\log P = \{61\}$, and scale 2^{45}. The circuit is multiplicative *depth one* (one plaintext-scalar multiplication and a rescale, with no relinearization and no bootstrapping), and decryption is N-out-of-N (no single party can decrypt). Communication figures are exact; timings are single-run wall-clock on one core of a commodity laptop CPU (Apple M4) and are reported as indicative. The circuit uses no ciphertext rotations and no relinearization, and because the only server operation is the one-shot aggregation, there is no encrypted training loop and hence no convergence behaviour to report under encryption. The server accumulates the weighted sum as a streaming reduction, holding at most two adapter-sized ciphertext sets at once (about $38\,\text{MiB}$), so its memory does not grow with N.

b) Communication.: A round is one upload and one download, and the protocol completes in a single round. The per-client traffic is fixed by the trainable-parameter count: a unit of up to 8192 parameters is a single $0.5\,\text{MiB}$ ciphertext. The headline configuration, a freeze-A rank-8 adapter with a classifier head (about 1.5×10^5 trainable parameters), occupies 19 ciphertexts, or $9.5\,\text{MiB}$ per client; freezing the down-projection is what halves this relative to training both factors. Total traffic scales linearly in the number of clients (table VI). The contrast with encrypted full-model schemes is structural:

TABLE VI
PER-CLIENT COMMUNICATION PER ROUND (MULTIPARTY CKKS, RING 2^{14}, 0.5 MiB PER CIPHERTEXT). A ROUND IS ONE UPLOAD AND ONE DOWNLOAD; TOTALS SCALE LINEARLY IN N (FIG. 3).

Trainable unit	ciphertexts	up=down per client
freeze-A adapter+head (RoBERTa)	**19**	9.5 **MiB**
freeze-A adapter+head (Qwen2.5-0.5B)	26	13 MiB
text head (4-class)	1	0.50 MiB
vision head (100-class)	10	5.0 MiB

TABLE VII
COMPUTATION TIME (MS, ONE CPU CORE, APPLE M4). DKG IS A ONE-TIME SETUP; THE ENCRYPT FIGURE IS PER CLIENT (CLIENTS ENCRYPT IN PARALLEL); AGGREGATION AND DECRYPTION SCALE LINEARLY IN $N\times$CIPHERTEXTS.

Trainable unit	N	DKG	encrypt/client	aggregate	decrypt
freeze-A adapter	10	11	40	76	44
freeze-A adapter	100	96	40	717	430
text head	10	13	2.3	3.9	2.4
text head	100	96	2.2	38.9	23.2
vision head	10	11	21.5	39.0	24.6
vision head	100	98	21.7	382.2	216.5

on the same backbone, encrypting the full RoBERTa-base (1.25×10^8 parameters) would move about 7.5 GiB per client *per round*, and such schemes run for many rounds, where HE-IFD encrypts only the adapter and runs once (fig. 3). The object stays small even on a billion-parameter backbone: a freeze-A adapter on the 0.5B Qwen2.5 model is 26 ciphertexts (13 MiB), because only the adapter is encrypted, never the backbone.

c) Computation.: The server's homomorphic work is tens of milliseconds at deployment scale and stays sub-second even at $N = 100$ (table VII). The measured operations follow simple laws: a client encrypts in about 2.1 ms per ciphertext (in parallel across clients), while the server aggregation and the threshold decryption scale linearly in $N\times$(ciphertexts/client), and the one-time distributed key generation grows linearly in N at about 1 ms per party. From these rates the freeze-A adapter (19 ciphertexts) costs about 40 ms to encrypt per client and, at $N = 100$, about 0.72 s to aggregate and 0.43 s to decrypt (table VII). Emitting k candidate aggregates (section III-G) costs k threshold decryptions, so a dozen candidates add about half a second; there is no bootstrapping anywhere in the protocol.

d) Comparison with other encrypted schemes.: Table VIII places HE-IFD against the encrypted federated-learning approaches. Two properties separate it from all of them. First, it is one-shot: it encrypts a single contribution and runs one round, where encrypted-training schemes such as POSEIDON [4] run multiparty CKKS for many rounds, hours of wall-clock even on small models, and aggregation schemes such as BatchCrypt, FedML-HE, and FedSHE [6], [7], [8] encrypt and sum updates on every round of an iterative protocol. Second, it encrypts only the small adapter and head, not the full model: the per-client object is 9.5 MiB against the gigabytes a full-model scheme moves on the same backbone, and there is no bootstrapping because the circuit

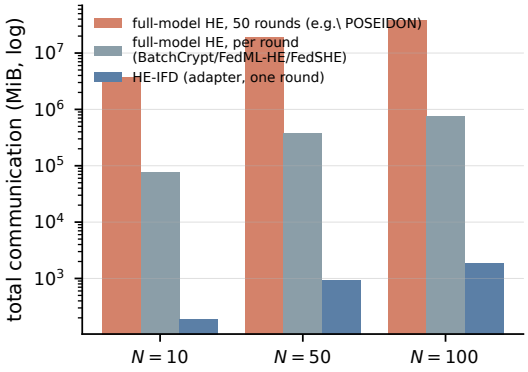

Fig. 3. Total communication on a shared backbone (RoBERTa-base), calculated for CKKS ring 2^{14} (0.5 MiB per ciphertext; 50 rounds for the iterative schemes). HE-IFD encrypts only the adapter and runs once; full-model encrypted schemes move gigabytes per round (BatchCrypt, FedML-HE, FedSHE) to terabytes over training (POSEIDON-style encrypted training).

is depth one. The closest recent scheme, SHE-LoRA [9], also encrypts only adapter parameters, but it is multi-round and pays its encryption cost on every round, and because it trains both adapter factors it carries the bilinear aggregation noise under encryption that our frozen-projection design removes. The secure-aggregation primitive of [5] is one-round but protects a per-round sum within an iterative protocol rather than a one-shot contribution. Figure 3 shows the per-round communication gap on a shared backbone, before the round-count multiplier that further separates a one-round protocol from a multi-round one.

The encrypted object stays depth one and the server cost grows only with the ciphertext count, never with the size of the frozen backbone the adapter sits in.

e) Correctness.: The decrypted aggregate matches the plaintext computation to a relative ℓ_2 error between 2.7×10^{-9} ($N = 10$) and 2.6×10^{-8} ($N = 100$). This is the encoding precision of CKKS at scale 2^{45}; it grows mildly with N as ciphertext additions and the key-switch smudging noise accumulate, and it stays several orders of magnitude below any tolerance the decoded model is sensitive to. Because the circuit is depth one, the error is not amplified.

f) Comparison with prior cryptographic federated learning.: The contrast with prior homomorphic federated learning is structural, not incremental. Encrypted-training methods such as POSEIDON [4] reach comparable accuracy but through many rounds of homomorphic computation that the source paper reports at roughly 1.4 hours of wall-clock; secure-aggregation primitives [5] sum updates once per round inside an iterative protocol. Our protocol runs in a single round whose server computation is milliseconds and whose communication is a few megabytes, because the only encrypted object is a bounded fine-tuning displacement of a small adapter, combined at multiplicative depth one. The broader method-level comparison is in table X.

G. Residual Leakage

The cryptographic guarantee of section III-I covers the contributions, not the final model, which every party obtains

TABLE VIII
HE-IFD AGAINST ENCRYPTED FEDERATED-LEARNING SCHEMES. COMMUNICATION IS PER CLIENT; HE-IFD'S IS CALCULATED FOR A FREEZE-A ADAPTER (ROBERTA-BASE, 19 CIPHERTEXTS), THE FULL-MODEL FIGURE FOR ENCRYPTING THE SAME BACKBONE.

Method	crypto	encrypted object	rounds	comm/client
HE-IFD (ours)	multiparty CKKS	adapter+head	**1**	9.5 **MiB**
SHE-LoRA [9]	CKKS (selective)	adapter subset	many	per round
POSEIDON	multiparty CKKS	full model	many	GiB ($\sim$1.4 h)
BatchCrypt / FedML-HE / FedSHE	CKKS	full model	many	GiB / round
Hyb-Agg	multi-key CKKS	update sum	per round	secure sum

in the clear after decryption. This is not a gap a stronger mechanism could close: any protocol that hands separate parties a shared, useful model must move knowledge between them, and a model that classifies categories a client never saw locally necessarily encodes information about the data that taught it those categories. The meaningful question is not whether residual leakage is zero, which it cannot be, but how large an attack surface a protocol exposes, and our design makes that surface as small as a one-shot federated method can. There is no alignment phase, so nothing data-derived is released before encryption, and the only object exposed in the clear is the single decrypted model.

Read as attack surfaces, the alternatives differ by orders of magnitude. Multi-round federated learning publishes a fresh update every round, so a curious server or peer accumulates a long sequence of views of each client over training [41], [42]. One-shot methods that rely on differential privacy ship, in a single object, essentially the whole of a client's local knowledge: a synthetic copy of its data, or a noised model. HE-IFD exposes only the final aggregate: the per-client displacements are encrypted and never appear in the clear, and because there is no alignment phase there is no prototype channel either.

We measure the residual on that one open channel with membership-inference attacks [43], [44], [45], training the target and a population of shadow models through the full federated pipeline and attacking the released model both with a loss-threshold test and with the likelihood-ratio attack LiRA, under an external adversary and under the *fellow-client* adversary that brings its own data as a prior, the strongest the threat model of section III-H admits (table IX). On three of the four tasks the leakage is at the level of random guessing: LiRA reaches an AUC of 0.49–0.50 and a true-positive rate of about 1% at a 1% false-positive rate, indistinguishable from chance. On the 77-class Banking77 it is mildly elevated, to an AUC of 0.59 and a 3.4% true-positive rate, the one task where coverage forces the model to memorise the most per class, and exactly the task where the accuracy gap to centralized is largest; even there the signal is small, and the fellow-client prior yields no measurable advantage over the external adversary. A participant therefore learns little about another's membership from the shared model beyond what any usable classifier of that data would reveal.

We do not claim the released model leaks nothing. A model that gives every client coverage of classes it never observed must carry information about the data that supplied that coverage, and an adversary with strong priors about a client's distribution can recover some of it; this is a property of useful shared models, not of our protocol in particular. What the protocol guarantees is that this residual is the *entire* surface (not multiplied across the rounds of an iterative protocol, and not handed over wholesale as released data), and driving the exposed surface down to that inherent floor, rather than promising to remove a leak that cannot be removed when parties set out to build a common model, is the privacy goal the design meets.

TABLE IX
MEMBERSHIP INFERENCE AGAINST THE RELEASED MODEL (LIRA, 16 SHADOW MODELS, THREE SEEDS). AUC AND THE TRUE-POSITIVE RATE AT A 1% FALSE-POSITIVE RATE, FOR AN EXTERNAL ADVERSARY AND A FELLOW CLIENT USING ITS OWN DATA AS A PRIOR. 0.50 AUC AND 1% TPR ARE CHANCE.

	external		fellow client	
Task (C)	AUC	TPR@1%	AUC	TPR@1%
AG-News (4)	0.50	0.9%	0.50	0.9%
TREC (6)	0.49	0.7%	0.49	0.7%
DBpedia (14)	0.50	0.9%	0.50	0.9%
Banking77 (77)	0.59	3.4%	0.59	3.4%

H. Comparison with Prior Work

Table X places the method among representative federated-learning approaches along the axes that matter here: whether the protocol is genuinely one-shot, what kind of privacy it provides on the contributions, how many rounds it needs, and its principal limitation relative to our goal. The one-shot distillation methods are either plaintext, and so offer no privacy on the contributions, or differentially private, and so pay a utility cost on the released model. The homomorphic-encryption methods either run for many rounds, as in encrypted training, or are secure-aggregation primitives that sum updates over an iterative protocol. The combination we occupy (a one-shot federated fine-tuning whose aggregation is protected by homomorphic encryption that adds no accuracy cost) has, to our knowledge, not been demonstrated before.

Two comparisons are the informative ones. Against POSEIDON, the closest cryptographic peer, the privacy guarantee is the same kind (encryption with no noise), but POSEIDON reaches its accuracy through multi-round encrypted *training* that the source paper reports at roughly 1.4 hours, whereas our protocol completes in a single round whose server computation is milliseconds and whose communication is tens of megabytes (section IV-F). Against the differentially private one-shot methods, the natural utility peers, the contrast is the central one: their accuracy is obtained under a privacy

TABLE X
COMPARISON WITH REPRESENTATIVE FEDERATED-LEARNING METHODS. REPORTED ACCURACIES ARE TAKEN VERBATIM FROM EACH SOURCE PAPER AT *its own* SETUP, SO THE COLUMN IS CONTEXT, NOT A LIKE-FOR-LIKE RANKING; THE PROTOCOL AXES (ONE-SHOT, PRIVACY, ROUNDS) ARE THE CONTROLLED COMPARISON. OURS IS THE ONLY METHOD THAT IS SIMULTANEOUSLY ONE-SHOT, FINE-TUNING-BASED, AND PROTECTED BY ENCRYPTION THAT ADDS NO ACCURACY COST.

Method	One-shot	Privacy	Rounds	Reported acc. (own setup)	Principal limitation vs. our goal
FedDF [28]	no	none	$\sim$100	rounds-to-target (CIFAR-10)	many rounds; no privacy
FuseFL [31]	no[†]	none	K blocks	84.3% (CIFAR-10)	K rounds; no privacy
DENSE [29]	yes	none	1	data-free (CIFAR-10)	no privacy on contributions
FedLPA [32]	yes	none	1	85.8% (MNIST)	no privacy; needs Fisher info
FedKT [36]	yes	DP-capable	1	90.5% (MNIST)	lossy when private
FedAUXfdp [34]	yes	DP (lossy)	1	75.2% (CIFAR-10, ε=0.5)	accuracy falls with budget
FedDiff [35]	yes	DP (lossy)	1	27.8% (CIFAR-10, ε=10)	accuracy falls with budget
POSEIDON [4]	no	HE (no noise)	many	89.9% (MNIST, $\sim$1.4 h)	multi-round; encrypted training
HE secure agg. [5]	no	HE	per round	protocol only	aggregation, not fine-tuning
HE-IFD (ours)	**yes**	**HE (no noise)**	**1**	**0.93/0.77 (DBpedia/Banking77)**	**none**

TABLE XI
HE-IFD AT THE PUBLISHED PARTITIONS OF PRIOR ONE-SHOT METHODS (FROZEN VIT-B/16 + FREEZE-A ADAPTER, THREE SEEDS). COMPARATOR NUMBERS ARE PAPER-VERBATIM AT THE CITED SETUP.

Matched setup	data, N, α	HE-IFD	reported
DENSE [29]	CIFAR-10, 5, 0.1	**0.96**	0.50
DENSE [29]	CIFAR-10, 5, 0.3	**0.96**	0.60
FedAUXfdp [34]	CIFAR-10, 20, 0.04	**0.94**	0.75[‡]
FedSD2C [33]	Tiny-ImageNet, 10, 0.1	**0.73**	—

[‡]FedAUXfdp at ε=0.5 on its CIFAR-10 setup.

budget that perturbs the released model, whereas encryption introduces no such perturbation.

[†]FuseFL's communication equals one-shot but it runs in K block-wise rounds.

I. Matched Setups and Scale

The accuracies in table X are each paper's own, at its own setup, and are context rather than a controlled comparison. To make the comparison controlled, we run HE-IFD at the *published partitions* of three one-shot methods, varying the dataset, client count, and Dirichlet α to match theirs while keeping our model class (a frozen ViT-B/16 with a freeze-A adapter, which we state). Table XI reports the result. At the data-free DENSE setup (CIFAR-10, $N = 5$) HE-IFD reaches 0.96 against their reported $0.50/0.60$; at the differentially private FedAUXfdp setup (CIFAR-10, $N = 20$, severe skew) 0.94 against 0.75 at ε=0.5; and at the FedSD2C setup (Tiny-ImageNet, $N = 10$) 0.73. The gains owe much to the stronger frozen backbone, which is the point: the protocol's cryptographic and communication structure carries unchanged onto a backbone that makes the task easy, where prior data-free and differentially private one-shot methods cannot reach the same accuracy and, in the DP case, pay an explicit budget for a weaker guarantee than encryption gives.

The protocol also carries to a billion-parameter-class backbone. On a frozen Qwen2.5-0.5B causal language model with a freeze-A adapter, HE-IFD reaches 0.87–0.88 on DBpedia, where the naive average collapses to 0.44, and 0.71–0.72 on AG-News, while the encrypted object stays at 26 ciphertexts (13 MiB): the cost is set by the adapter, not by the 0.5B backbone behind it, which is never encrypted.

J. Beyond Pretrained Backbones

The protocol does not require a pretrained backbone; it requires a shared frame. Where clients instead train a small network from a common random initialization, the same one-shot encrypted aggregation applies, but the shared frame must be established rather than inherited. Without a pretrained representation the clients' bounded updates are expressed in a coordinate system fixed only by the random seed, so agreeing on a useful starting point reintroduces the kind of alignment step a pretrained backbone removes. We therefore present pretrained fine-tuning as the primary setting and treat the from-scratch variant as a generalization whose alignment cost is left to future work.

K. Scope and Limitations

Two boundaries follow from well-understood properties rather than incidental shortcomings. First, the single linear aggregation pays a gap to centralized fine-tuning that widens with the size of the label space under severe skew, because a client contributes signal only for classes it holds and the shared head is untrained; this is an inherent property of combining bounded one-shot updates without an alignment phase, largest on the 77-class Banking77, and the coverage-weighted candidate contains it (to a gap of 0.11) rather than removing it. Second, the method fine-tunes a pretrained backbone, so it inherits that backbone's fit to the task and applies to the label-groundable foundation models used for transfer today rather than to a backbone with no relevant pretraining. Both are stated where they apply; neither touches the privacy claim, which holds regardless of the accuracy the shared model reaches.

a) Malicious and colluding clients.: Our cryptographic guarantees assume honest-but-curious participants. An actively malicious client could upload a crafted displacement to bias the shared model, and because the contributions are encrypted the server cannot inspect them, so plaintext outlier filtering does not apply. The leave-one-out candidates of section IV-C give a lightweight client-side defense that already neutralises a single poisoning client without any server-side filtering, but it assumes an honest-majority holdout and does not cover colluding or adaptive adversaries. Stronger guarantees compatible with the encryption are left to future work: an

upload-time norm bound enforced by a zero-knowledge proof, which caps any single client's influence without revealing its update, and encrypted-domain robust aggregation evaluated homomorphically. The one-shot structure shrinks the attack surface to a single round but removes the cross-round consistency checks a multi-round protocol could use, and reconciling robust aggregation with depth-one encryption is the open problem this direction must solve.

V. Conclusion

We presented HE-IFD, a one-shot federated learning protocol in which the server's only operation on client contributions is a linear aggregation computed under multiparty homomorphic encryption at multiplicative depth one. The construction gives the contributions cryptographic privacy at no accuracy cost and at the communication budget of a single round; because the backbone is frozen and stays outside the encrypted combination, the protocol and its cost are independent of the model behind it. What makes a one-shot linear aggregation succeed is a co-design: every client begins from the same public pretrained backbone and a standard adapter initialization, so the bounded displacements they upload are expressed in a common frame and add constructively, and freezing the adapter's down-projection makes that aggregation exact task arithmetic rather than an approximation, which is what lets it run at depth one under encryption. Because a blind server cannot adapt the aggregation rule to the data, we let the server emit several depth-one candidate aggregates and the clients select among them after decryption, which lifts the released model under heterogeneity and yields a measured defense against a poisoning client, at no homomorphic cost. Across vision and language tasks, from a frozen RoBERTa and ViT to a billion-parameter language model, the method yields a common model stronger than any client's own and, outside the most skewed regimes, close to a centrally fine-tuned one; a multiparty CKKS implementation reproduces the encrypted aggregation within the scheme's approximation bounds at a few megabytes per round; and membership inference against the released model stays at or near chance. Extending the protocol to colluding or adaptive malicious participants, where verifiable homomorphic encryption [62], [63] could certify that the server computed the aggregation honestly, is the natural direction for future work.

Acknowledgments

We acknowledge the support of TÜBİTAK (the Scientific and Technological Research Council of Türkiye) projects 124E091 and 124N941. The authors used Claude Sonnet 4.6 and Claude Opus 4.6 to enhance the manuscript by shortening sentences, correcting typos, and improving grammar.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[2] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *ACM CCS*, 2017, pp. 1175–1191.

[3] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 308–318. [Online]. Available: https://arxiv.org/abs/1607.00133

[4] S. Sav, A. Pyrgelis, J. R. Troncoso-Pastoriza, D. Froelicher, J.-P. Bossuat, J. Sá Sousa, and J.-P. Hubaux, "POSEIDON: Privacy-preserving federated neural network learning," in *NDSS*, 2021.

[5] Kemmaka and Tran, "Hyb-Agg: Communication-efficient secure aggregation via hybrid multi-key homomorphic encryption and masking," *arXiv:2511.23252*, 2025.

[6] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "Batchcrypt: Efficient homomorphic encryption for cross-silo federated learning," in *USENIX ATC*. USENIX Association, 2020, pp. 493–506.

[7] W. Jin, Y. Yao, S. Han, J. Gu, C. Joe-Wong, S. Ravi, S. Avestimehr, and C. He, "Fedml-he: An efficient homomorphic-encryption-based privacy-preserving federated learning system," in *NeurIPS*, vol. 36, 2023.

[8] X. Wei, R. Huang, L. Lei, and J. Wu, "Fedshe–cq: A communication-efficient homomorphic encryption framework for federated learning," *Computer Networks*, vol. 260, p. 111813, 2025.

[9] J. Li *et al.*, "SHE-LoRA: Selective homomorphic encryption for federated tuning with heterogeneous LoRA," *arXiv preprint arXiv:2505.21051*, 2025.

[10] K. Garimella, N. K. Jha, and B. Reagen, "Sisyphus: A cautionary tale of using low-degree polynomial activations in privacy-preserving deep learning," in *PPML Workshop, CCS*, 2021.

[11] M. Baruch, N. Drucker, L. Greenberg, and G. Moshkowich, "A methodology for training homomorphic encryption friendly neural networks," in *ACNS Workshops*, ser. Lecture Notes in Computer Science, vol. 13285. Springer, 2022, pp. 536–553.

[12] P. Agamennone, "Polynomial approximations of relu for secure cnn inference in homomorphic encryption environments," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E108.A, no. 7, 2025.

[13] F. Al Hossain and T. Rahman, "A training framework for optimal and stable training of polynomial neural networks," *arXiv preprint arXiv:2505.11589*, 2025.

[14] A. Ibarrondo and M. Önen, "FHE-compatible batch normalization for privacy-preserving deep learning," in *Privacy in Machine Learning Workshop (PriML), NeurIPS*, 2021. [Online]. Available: https://www.eurecom.fr/en/publication/5626

[15] J. Zhang, S. Vahidian, M. Kuo, C. Li, R. Zhang, T. Yu, G. Wang, and Y. Chen, "Towards building the federated GPT: Federated instruction tuning," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, arXiv:2305.05644 (FedIT).

[16] Z. Wang, Z. Shen, Y. He, G. Sun, H. Wang, L. Lyu, and A. Li, "FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2409.05976.

[17] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2402.11505 (FlexLoRA).

[18] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, and G. Joshi, "Heterogeneous LoRA for federated fine-tuning of on-device foundation models," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, arXiv:2401.06432.

[19] P. Guo, S. Zeng, Y. Wang, H. Fan, F. Wang, and L. Qu, "Selective aggregation for low-rank adaptation in federated learning," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.01463 (FedSA-LoRA).

[20] Y. Sun, Z. Li, Y. Li, and B. Ding, "Improving LoRA in privacy-preserving federated learning," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2403.12313.

[21] G. Ilharco, M. T. Ribeiro, M. Wortsman, L. Schmidt, H. Hajishirzi, and A. Farhadi, "Editing models with task arithmetic," in *ICLR*, 2023.

[22] Z. Tao, I. Mason, S. Kulkarni, and X. Boix, "Task arithmetic through the lens of one-shot federated learning," *Transactions on Machine Learning Research (TMLR)*, 2025, arXiv:2411.18607.

[23] Anonymous, "Revisiting federated fine-tuning: A single communication round is enough for foundation models," *arXiv preprint arXiv:2412.04650*, 2024.

[24] N. Guha, A. Talwalkar, and V. Smith, "One-shot federated learning," *arXiv preprint arXiv:1902.11175*, 2019. [Online]. Available: https://arxiv.org/abs/1902.11175

[25] J. Wang *et al.*, "Towards one-shot federated learning: Advances, challenges, and future directions," *arXiv preprint arXiv:2505.02426*, 2025.

[26] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," *arXiv preprint arXiv:1910.03581*, 2019. [Online]. Available: https://arxiv.org/abs/1910.03581
[27] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, "DS-FL: Distillation-based semi-supervised federated learning for medical image classification," in *IEEE ICC*, 2021.
[28] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *NeurIPS*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/18df51b97ccd68128e994804f3eccc87-Abstract.html
[29] J. Zhang, C. Chen, B. Li, L. Lyu, S. Wu, S. Ding, C. Shen, and C. Qi, "DENSE: Data-free one-shot federated learning," in *NeurIPS*, vol. 35, 2022, pp. 21 414–21 428. [Online]. Available: https://arxiv.org/abs/2112.12371
[30] R. Dai, Y. Zhang, A. Li, T. Liu, X. Yang, and B. Han, "Enhancing one-shot federated learning through data and ensemble co-boosting," in *ICLR*, 2024.
[31] Z. Tang, Y. Zhang, P. Dong, Y.-m. Cheung, A. C. Zhou, B. Han, and X. Chu, "Fusefl: One-shot federated learning through the lens of causality with progressive model fusion," in *NeurIPS*, 2024.
[32] X. Liu, L. Liu, F. Ye, Y. Shen, X. Li, L. Jiang, and J. Li, "FedLPA: One-shot federated learning with layer-wise posterior aggregation," in *NeurIPS*, 2024.
[33] Y. Zhang *et al.*, "Federated one-shot learning with data-free distillation," in *NeurIPS*, 2024.
[34] H. Hoech, R. Rischke, K. Mueller, and W. Samek, "FedAUXfdp: Differentially private one-shot federated distillation," in *IJCAI Workshop on Federated Learning*, 2022.
[35] M. Mendieta, G. Sun, and C. Chen, "Navigating heterogeneity and privacy in one-shot federated learning with diffusion models," in *WACV*, 2025, pp. 2601–2610. [Online]. Available: https://openaccess.thecvf.com/content/WACV2025/html/Mendieta_Navigating_Heterogeneity_and_Privacy_in_One-Shot_Federated_Learning_with_Diffusion_Models_WACV_2025_paper.html
[36] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," in *IJCAI*, 2021, pp. 1484–1490.
[37] L. Sun and L. Lyu, "Federated model distillation with noise-free differential privacy," in *IJCAI*, 2021, pp. 1563–1569.
[38] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *NeurIPS*, vol. 32, 2019.
[39] D. I. Dimitrov, M. Baader, and M. Vechev, "SPEAR: Exact gradient inversion of batches in federated learning," in *NeurIPS*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/c13cd7feab4beb1a27981e19e2455916-Abstract-Conference.html
[40] V. Carletti, P. Foggia, C. Mazzocca, G. Parrella, and M. Vento, "SoK: Gradient inversion attacks in federated learning," in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/carletti
[41] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *IEEE S&P*, 2017, pp. 3–18. [Online]. Available: https://arxiv.org/abs/1610.05820
[42] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *IEEE S&P*, 2019, pp. 739–753.
[43] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *31st IEEE Computer Security Foundations Symposium (CSF)*, 2018, pp. 268–282. [Online]. Available: https://arxiv.org/abs/1709.01604
[44] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Network and Distributed System Security Symposium (NDSS)*, 2019. [Online]. Available: https://arxiv.org/abs/1806.01246
[45] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer, "Membership inference attacks from first principles," in *IEEE S&P*, 2022, pp. 1897–1914. [Online]. Available: https://arxiv.org/abs/2112.03570
[46] R. Kerkouche, G. Mema, M. Fritz, J. Ramon, and N. Samarin, "Client-specific property inference against secure aggregation in federated learning," in *IEEE S&P*, 2023. [Online]. Available: https://arxiv.org/abs/2303.03908
[47] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015, pp. 1322–1333. [Online]. Available: https://dl.acm.org/doi/10.1145/2810103.2813677
[48] B. Hitaj, G. Ateniese, and F. Pérez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *ACM CCS*, 2017, pp. 603–618.
[49] H. Takahashi, J. Liu, and Y. Liu, "Breaching FedMD: Image recovery via paired-logits inversion attack," in *CVPR*, 2023, pp. 12 198–12 207.
[50] Z. Yang, Y. Zhao, and J. Zhang, "FD-Leaks: Membership inference attacks against federated distillation learning," in *APWeb-WAIM*, ser. Lecture Notes in Computer Science, vol. 13423. Springer, 2023, pp. 364–378.
[51] X. Wang, L. Wu, and Z. Guan, "GradDiff: Gradient-based membership inference attacks against federated distillation with differential comparison," *Information Sciences*, vol. 658, p. 120068, 2024.
[52] M. Jagielski, M. Nasr, K. Lee, C. Choquette-Choo, N. Carlini, and F. Tramèr, "Students parrot their teachers: Membership inference on model distillation," in *NeurIPS*, vol. 36, 2023.
[53] Y. Gu and Y. Bai, "LDIA: Label distribution inference attack against federated learning in edge computing," *Journal of Information Security and Applications*, vol. 74, p. 103475, 2023. [Online]. Available: https://dl.acm.org/doi/10.1016/j.jisa.2023.103475
[54] H. Shi, T. Ouyang, and A. Wang, "Unveiling client privacy leakage from public dataset usage in federated distillation," *PoPETs*, vol. 2025, no. 4, pp. 201–215, 2025.
[55] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *ASIACRYPT*, 2017, pp. 409–437.
[56] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *Journal of the ACM*, vol. 60, no. 6, pp. 43:1–43:35, 2013.
[57] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology – EUROCRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 10820. Springer, 2018, pp. 360–384. [Online]. Available: https://eprint.iacr.org/2018/153
[58] C. Mouchet, J. R. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty homomorphic encryption from ring-learning-with-errors," *PoPETs*, vol. 2021, no. 4, pp. 291–311, 2021.
[59] J. So, R. E. Ali, B. Güler, J. Jiao, and A. S. Avestimehr, "Securing secure aggregation: Mitigating multi-round privacy leakage in federated learning," in *AAAI*, vol. 37, no. 8, 2023, pp. 9925–9933.
[60] R. Kerkouche, G. Ács, and M. Fritz, "Client-specific property inference against secure aggregation in federated learning," in *WPES*. ACM, 2023, pp. 15–30.
[61] C. Mouchet, J.-P. Bossuat, J. Troncoso-Pastoriza, and J.-P. Hubaux, "Lattigo v5: A multiparty homomorphic encryption library in Go," in *WAHC*, 2021.
[62] A. Viand, C. Knabenhans, and A. Hithnawi, "SoK: Fully homomorphic encryption compilers and verifiable computation," in *IEEE S&P*, 2023. [Online]. Available: https://arxiv.org/abs/2301.07041
[63] S. Atapoor, K. Baghery, H. V. L. Pereira, and J. Spiessens, "Verifiable FHE via lattice-based SNARKs," Cryptology ePrint Archive, Paper 2024/582, 2024. [Online]. Available: https://eprint.iacr.org/2024/582

HALİL İBRAHİM KANPAK received his B.Sc. degree in Mathematics from Koç University. He is currently pursuing his Ph.D. at Koç University, where he is a member of the Cryptography, Cyber Security & Privacy Research Group. His research interests include cryptography and related areas in Deep Learning.

Asst. Prof. SİNEM SAV received her Ph.D. in Computer and Communication Sciences from École Polytechnique Fédérale de Lausanne (EPFL). She also holds M.Sc. and B.Sc. degrees in Computer Engineering from Bilkent University, where she currently serves as an Assistant Professor in the Computer Engineering department. Her research interests include privacy-preserving machine learning and applied cryptography.

Prof. ALPTEKİN KÜPÇÜ received his Ph.D. degree from Brown University Computer Science Department in 2010. Since then, he has been working as a faculty member at Koç University, leading the Cryptography, Cyber Security & Privacy Research Group that he founded. He is an ACM and IEEE Senior Member, and a co-founder of Xtinge Technology Inc.